DS	**Session**	Tree 1	**Question Information**	Level 1 Challenge 63

Question description

You are given an n×n grid representing the map of a forest. Each square is either empty or contains a tree. The upper-left square has coordinates (1,1), and the lower-right square has coordinates (n,n).

Your task is to process q queries of the form: how many trees are inside a given rectangle in the forest?

Constraints

- 1≤n≤1000
- 1≤q≤2·10^5
- 1≤y1≤y2≤n
- 1≤x1≤x2≤n

Input

The first input line has two integers n and q: the size of the forest and the number of queries.
Then, there are n lines describing the forest. Each line has n characters: . is an empty square and * is a tree.
Finally, there are q lines describing the queries. Each line has four integers y1, x1, y2, x2 corresponding to the corners of a rectangle.

Output
Print the number of trees inside each rectangle.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
4 3
.*..
*.**
**..
****
2 2 3 4
3 1 3 1
1 1 2 2
```

EXPECTED OUTPUT

```
3
1
2
```

Test Case 2

INPUT (STDIN)

```
4 3
.*.*
*.**
**.*
***.
1 2 3 4
3 2 3 4
1 4 2 3
```

EXPECTED OUTPUT

```
6
2
0
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
for(i=1;i<=n;i++)
```

Complexity Test Cases

Test Case 1

CYCLOMATIC COMPLEXITY

8

Test Case 2

TOKEN COUNT

310

Test Case 3

NLOC

28

Code

solution.c

```c
#include <stdio.h>

int pref[1005][1005];

int main() {
    int n, q;
    if (scanf("%d %d", &n, &q) != 2) return 0;
    for (int i = 1; i <= n; i++) {
        char s[1005];
        scanf("%s", s);
        for (int j = 1; j <= n; j++) {
            int val = (s[j - 1] == '*') ? 1 : 0;
            pref[i][j] = val + pref[i - 1][j] + pref[i][j - 1] - pref[i - 1][j - 1];
        }
    }
    while (q--) {
        int y1, x1, y2, x2;
        scanf("%d %d %d %d", &y1, &x1, &y2, &x2);
        int ans = pref[y2][x2] - pref[y1 - 1][x2] - pref[y2][x1 - 1] + pref[y1 - 1][x1 - 1];
        printf("%d\n", ans);
    }
    return 0;
}
```

Output

```
Problems  Output  Debug Console  Terminal  Ports                powershell

PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
4 3
.*..
*.**
**.*
****
2 2 3 4
3 1 3 1
1 1 2 2
3
1
2
PS C:\C Programming>
```